Resync-and-Anchor Copy Drafting for Lossless Code Fast-Apply

Novain Shibu (independent) · October 2026 · Technical report · Code and data: `resync-apply` (Apache-2.0) · DOI: *pending*

Abstract

AI code editors apply an edit by having a model rewrite the whole file. Most of that output is copied from the original, so speculative decoding can propose long copied spans and let the model verify them in one forward pass, with output equal to greedy decoding. We study how few verification passes a copy drafter can get away with. Our drafter verifies several copy branches per pass as one token tree (7.4 per pass on average, at most 15). The candidates come from four sources: *resync* positions after a mismatch (the model replaced, inserted or skipped one source token), *1-token anchors* ranked by distance, read frontiers, and branch lengths taken from a learned table.

We tested it on 199 real edits from the Kortix FastApply dataset with the open Kortix/FastApply-1.5B model. It needs **~1.27x fewer verification passes than tuned SuffixDecoding**, the strongest open method and the one built into vLLM. On the 169 edits never used for tuning the ratio is **1.26–1.28x (bootstrap 95% CI 1.19–1.36)**. On a free NVIDIA T4 in a Hugging Face Python loop the pass reduction did **not** turn into a clear speed-up: 1.05x lower wall-clock on the 30 tuning edits (56.8 s vs 59.9 s) and **1.00x on 29 held-out edits** (56.7 s vs 56.7 s, despite 1.37x fewer passes). Both methods are **24–37x faster than normal decoding** (first 5 edits of each run). Output matched greedy decoding on every edit compared (Section 6.4). We also report seven ideas that did not work, a loss breakdown against a copy oracle, and a laptop simulator that reproduces our GPU pass counts exactly (664, 644 and 603 passes in three GPU runs).

1. Introduction

"Fast apply" is the step where an AI code editor turns a short edit snippet into a complete updated file. Cursor popularised doing this with a dedicated model plus "speculative edits": the original file serves as the draft. Commercial services (Morph, Relace) and OpenAI's Predicted Outputs offer similar features. Open-source servers rely on model-free drafters: prompt lookup decoding (n-gram matches into the prompt) or SuffixDecoding (Snowflake Arctic Inference; a suffix tree over the prompt and earlier outputs, with adaptive draft length), which ships in vLLM.

When the model's output stops matching the current copy position, a copy drafter must find where to copy from next. Every wrong guess costs one full forward pass. This report measures that cost and shows that a small set of cheap, rule-based candidates removes about a fifth of all passes compared with tuned SuffixDecoding, and why that did not become a wall-clock gain on a T4. The contributions:

1. A multi-branch copy drafter with resync candidates and 1-token anchors, evaluated against tuned SuffixDecoding on a real fast-apply model and dataset, including a held-out split.
2. A laptop simulator that reproduces GPU pass counts exactly and wall-clock within ~1–2 s. Each idea is scored in seconds.
3. Negative results for seven ideas, and a breakdown of the remaining gap to a copy oracle.

2. Related work

- **Copy and retrieval drafting.** LLMA / Inference with Reference (2023) copies the reference span with the longest matching prefix. Prompt lookup decoding (2023) does n-gram matching into the prompt. REST (retrieval trie + tree verification). SAM Decoding (suffix automaton, longest match). Graft (2026) fills pruned draft-tree slots with retrieved candidates. SuffixDecoding (Snowflake) uses suffix trees with adaptive speculation length and an optional tree mode.
- **Code editing.** Cursor speculative edits (closed). EfficientEdit (ASE 2025): reuses the original code as a linear draft, re-anchors by matching the tail of the generated text to a prefix of the remaining original code, and uses an edit-oriented draft model for new text (10–13x vs autoregressive decoding; it does not compare against SuffixDecoding). QiMeng-PRepair (2026). An empirical study of speculative decoding on software-engineering tasks (2026) finds SuffixDecoding strongest on editing and repair.
- **Reuse of rejected predictions and trees.** Token Recycling, Lookahead/Jacobi decoding, SpecInfer/Medusa tree verification, FlashInfer cascade attention.

What we believe is not in prior work. We read the papers above and EfficientEdit's released code. None of them tries explicit resync positions after a one-token substitution, insertion or deletion as parallel branches. None combines them with 1-token anchors and learned branch sizing, or measures this against SuffixDecoding on fast apply. To our knowledge this is new as a combination. The individual parts (copying, n-gram anchors, tree verification) are known.

3. Pass physics on a T4

Each verification pass is one forward pass of the target model over the drafted tokens. We measured its cost for FastApply-1.5B (fp16, T4) with a 2,048-token context:

drafted tokens	1	8	32	64	128	256	512	1024	2048
ms per pass	38.7	35.7	37.1	37.0	39.2	59.6	108.6	215.2	463.1

The cost is flat up to ~127 drafted tokens, then rises by ~0.2 ms per token. The Hugging Face Python loop adds ~22 ms per pass. Our tree-mask construction adds ~16 ms on tree passes. For our best design on 30 edits the time splits as prompt prefill 20%, fixed per-pass cost 69%, extra drafted tokens 11%. **Passes are the currency:** removing a pass saves ~60 ms, while drafting 100 extra tokens costs almost nothing below the knee.

4. Method

The drafter (`sim/drafter.py`) is one Python class. The CPU simulator and the GPU loop both run it.

1. **Index.** The prompt (original file + edit snippet) is indexed by n-grams for n = 32, 16, 12, 8, 6, 4, 3, 2, 1.
2. **Main branch.** Continue copying at the current pointer. The draft length doubles after a fully accepted pass and shrinks to 2 × (accepted tokens) + 16 after a miss (bounded by kmin/kmax).
3. **Resync candidates (after a mismatch).** If the model emitted token *t* where the source had *s[p]*, propose copying from *p+1* (substitution), *p* (insertion: the model added *t*) and *p+2* (deletion). If *s[p+1] = t*, the deletion branch is tried first.
4. **Anchors.** Positions after the longest n-gram matches of the latest output, longest n first, down to **1-token anchors**. Ties are ranked nearest-after the previous pointer.

5. **Read frontiers.** The furthest position copied so far in the original code and in the snippet, and the next line start after each.
6. **Tree verification.** All branches (up to 3 resync + 4 frontier + K = 8 anchor branches, deduplicated; 7.4 per pass on average in the final configuration, counting single-branch passes) are verified together as one token tree (4D attention mask). The KV cache keeps only the accepted path. Acceptance is greedy: a drafted token is kept only if it equals the model's argmax.
7. **Learned branch sizing.** An expected-copy-length table over candidate features (kind, n-gram strength, distance bucket, region, line start), trained on records 100–198. It cuts side-branch lengths (1.0 × expected, minimum 8) and caps the main branch (6 × expected).
8. **Hybrid passes.** A plain single-branch pass when the previous pass was fully accepted, a tree only after a miss.
9. **Fallback.** With no anchor, copy what followed the latest earlier match in the model's own output.

5. Experimental setup

- **Model:** Kortix/FastApply-1.5B-v1.0 (fine-tuned Qwen2.5-Coder-1.5B), fp16, batch size 1, greedy, Hugging Face transformers. **Data:** Kortix/FastApply-dataset-v1.0 test split, 199 edits. Prompts average 1,471 tokens (max 4,679). Outputs total 229,173 tokens. Model and data are both Apache-2.0.
- **Baseline:** SuffixDecoding from arctic-inference 0.3.0 in the same loop, tuned in its favour: max_spec_tokens 256, max_spec_factor 4.0, min_token_prob 0.1, and one cache shared across requests, so it also learns from earlier edits' outputs (our drafter does not). Tree mode (`use_tree_spec`) was also measured.
- **Simulator.** Greedy verification is deterministic given the true output. Pass counts for any drafter can therefore be computed on a CPU from the recorded outputs (`traces/oracle.json`; produced with greedy verification and equal to plain `generate` on the 10 edits checked). Simulated SuffixDecoding passes equal the real GPU count (868 on the first 30 edits). Our drafters give 664 simulated = 664 on GPU, and 644 = 644 (tuning edits) and 603 = 603 (held-out edits) for the final configuration. Wall-clock is predicted from the measured cost curve plus the 22 ms loop and 16 ms tree costs. The tree run was predicted at 69.5 s and measured at 70.5 s.
- **Splits.** The first 30 edits have GPU wall-clock for every method and were used for tuning. Edits 30–198 were never used for tuning. The sizing table was trained on 100–198, so the strictly clean held-out subset for the learned variant is 30–99. The untuned and time-tuned *rule* configurations use no table, so 30–198 is held out for them.
- **Hardware:** one free Kaggle NVIDIA T4 (16 GB).

6. Results

6.1 Verification passes (simulator, exact)

Configuration	Edits	Ours	SuffixDecoding linear	SuffixDecoding tree
untuned (K=8, strong 64, weak 16)	first 30 (tuning set)	664	868 (1.31x)	843 (1.27x)
untuned	**held-out 30–198**	**3,314**	4,170 (**1.26x**)	4,105 (1.24x)
time-tuned (K=8, s32, w8, kmin 64, kmax 384)	**held-out 30–198**	**3,253**	4,170 (**1.28x**)	4,105 (1.26x)

both	all 199	3,915–3,978	5,038 (1.27–1.29x)	4,948 (1.24–1.26x)
learned sizing + hybrid (final)	first 30 (tuning set)	644	868 (1.35x)	843 (1.31x)
learned sizing + hybrid (final)	**held-out 30–99**	**1,370**	1,774 (**1.29x**)	1,763 (1.29x)

Bootstrap 95% confidence intervals for the held-out ratio vs SuffixDecoding linear: **1.19–1.33** (untuned) and **1.21–1.36** (time-tuned), and **1.22–1.37** for the final configuration on 30–99 (its table was trained on 100–198, so 30–99 is its only clean split). On the first 30 edits the per-edit result was 14 wins, 11 losses and 5 ties (bootstrap CI of the ratio 1.14–1.47). The gain is concentrated in a few large files.

6.2 Ablation (passes; first 30 / all 199)

Drafter	first 30	all 199
single pointer, n-gram re-anchor	979	5,708
+ resync candidates	852	5,144
+ K=4 side branches (32 tokens)	794	4,785
+ 1-token anchors, K=8	689	4,175
+ read frontiers	680	4,138
+ strength-weighted branch lengths	**664**	**3,978**
+ learned sizing + hybrid (GPU run)	644	–
SuffixDecoding linear / tree (reference)	868 / 843	5,038 / 4,948

Resync gives the largest single step on the first 30 (979 → 852, −13%; all 199: 5,708 → 5,144, −10%). 1-token anchors give the second largest (794 → 689; all 199: 4,785 → 4,175). Rows 1–5: `sim/ablation.py`.

6.3 Wall-clock (T4, 30 edits, Hugging Face loop)

Each row's ratio is against SuffixDecoding measured in the same job (59.9–66.5 s across jobs; Kaggle T4s vary).

Method	Passes	Time	Identical	vs SuffixDecoding
Tree v1 (Python O(q²) mask)	664	160.2 s	30/30	0.42x
Tree, fast numpy mask, time-tuned	662	70.5 s	30/30	0.88x
Learned sizing + hybrid	**644**	**56.8 s**	**30/30**	**1.05x**
Learned sizing + hybrid + cascade attention	644	61.3 s	30/30	0.98x
SuffixDecoding (tuned)	869	59.9 s	30/30	1.00x

SuffixDecoding takes 869 passes here but 868 in the simulator and in the first GPU run. The GPU timing jobs send one warm-up request into SuffixDecoding's shared cache before the 30 edits, which slightly changes its contents. The outputs were identical, so the one-pass difference comes from cache state, not from the model.

"Identical" = equal to plain `generate` on edits 1–5 and to the other methods' output on edits 6–30 (Section 6.4).

Held-out wall-clock run. We then ran the final configuration against SuffixDecoding on dataset rows 30–59, which were never used for tuning (29 edits; one over-long row is skipped by the same rule as in every run):

Method (held-out rows 30–59)	Passes	Time	Identical	vs SuffixDecoding
Learned sizing + hybrid	**603**	**56.7 s**	**29/29**	**1.00x**
SuffixDecoding (tuned)	824	56.7 s	29/29	1.00x

Passes stay 1.37x lower, but wall-clock is a tie. Our tree passes cost more than SuffixDecoding's linear passes (Section 3 and Section 9), and on this hardware and loop that cancels the saved passes. The simulator predicted our 603 passes exactly. SuffixDecoding's simulated count for the same edits is 816 (its cache there already holds edits 0–29), against 824 on the GPU (empty cache plus warm-up).

Normal greedy decoding of the first 5 edits (6,914 tokens) took 342.8 s. On the same 5 edits our method took 9.2 s (**37x**) and SuffixDecoding 9.9 s (35x). On the first 5 held-out edits (3,828 tokens) normal decoding took 158.9 s, and both methods took 6.5 s (**24x**). An earlier run on the first 10 edits measured 20.8x for our older single-branch drafter and 23.3x for SuffixDecoding. **On a T4 in this loop, a 1.35–1.37x pass advantage becomes a 1.00–1.05x time advantage**, because tree passes cost more (Section 3).

6.4 Output equality

All methods use greedy verification and so should reproduce greedy decoding. Two checks were made. The output was compared with plain `generate` on 10 edits (oracle run, 10/10 equal), 5 edits (race run, 5/5) and 5 held-out edits (held-out run, 5/5). On the remaining edits of each timing run all methods produced identical outputs (30/30 and 29/29). We therefore say "matched greedy on every edit compared", not "bit-identical". Rare fp16 near-ties can flip an argmax for any method that changes the batch shape. In earlier runs with another model and dataset (Qwen2.5-Coder-1.5B-Instruct on 48 CanItEdit items), 1–2 items differed from plain `generate` for every method, prompt lookup and SuffixDecoding included.

7. What did not work

Rows 1–3 were measured against the dataset's reference files on 200 edits, before the model-output traces existed; the others use the traces or GPU runs.

Idea	Result
Merge-drafter (deterministically merge original + snippet, verify the predicted file)	1.04x fewer passes than SuffixDecoding. The merge was exact for only 7 of 200 files
Multi-candidate n-gram matching (K up to 16)	1.09x vs SuffixDecoding tree
Structural jump candidates (start of next new snippet line)	1.09x, no gain over multi-candidate
Jacobi residue recycling (reuse predictions after the mismatch)	residue branch won 18 of 654 passes. Divergences are mostly insertions, not substitutions
Learned ranker for candidate order	~1%
Junction forks from a line-level diff	~1% (644 → 638)
Cascade attention (exact log-sum-exp merge)	exact on 4/4 test trees but slower on T4 (61.3 s vs 56.8 s)
Small helper draft model for new text (projection)	new-text passes emit 0.9% of tokens; ceiling 1.02–1.12x fewer passes before paying for the model

8. Remaining headroom

Copy oracle. An oracle that always copies the longest matching span (it knows the future output) needs 374 passes on the first 30 edits (2,118 on all 199), against our 638–664. We classified our passes:

- 249 were as good as the oracle;
- 161 were genuinely new text;
- 191 had the right position among the branches, but the branch was too short;
- 37 lacked the right candidate.

Making all branches long only reaches 592–600 passes, and on a T4 the extra tokens raise predicted time from 65 s to 73–104 s. The rest of the oracle's advantage comes from knowing which of many matches is right, which no cheap rule found.

Other hardware (projection, NOT measured). We replaced the measured curve with first-principles cost models. The predicted ratio vs SuffixDecoding:

- T4 with no loop overhead and a fused tree pass: ~1.10x
- 7B model on 2×T4: ~0.89x (we lose: the flat region shrinks and our extra tree tokens cost)
- H100 with a vLLM-like loop: ~1.22x (1.5B) and ~1.27x (7B)

These numbers only show which way the result should move. Measuring them is future work.

9. Limitations

- One model (1.5B), one dataset, one GPU type, batch size 1, greedy decoding only.
- **No wall-clock gain on held-out edits on a T4** (1.00x). The 1.05x on the tuning edits should be read as noise-level.
- The final configuration verifies 2.3 drafted tokens per output token (137 per pass). SuffixDecoding verifies 1.48 (67 per pass; linear and tree modes alike), so we check ~1.6x more tokens per output token. Under heavy batching the GPU is no longer flat in drafted tokens, and the gain may shrink or reverse.
- Wall-clock was measured in a Hugging Face Python loop, not in vLLM or SGLang.
- Settings were tuned on the first 30 edits. Pass counts and wall-clock were re-checked on held-out edits (Sections 6.1 and 6.3).
- Output equality is shown on the edits compared (Section 6.4), not proven.

10. Reproducibility

- `python reproduce.py` (laptop, ~1 minute, no GPU) prints 3,978 / 664 passes for the drafter, 2,118 / 374 for the copy oracle, and 644 for the configuration of the GPU wall-clock run. `python sim/heldout_split.py` prints every split with bootstrap intervals, using SuffixDecoding's per-edit counts in `traces/suffix_per_record.json`. `python sim/hybrid.py` predicts T4 wall-clock from the measured curve.
- `traces/oracle.json`: 199 tokenized prompts, the model's outputs, the original GPU timings and the cost curve.
- `kaggle/`: the scripts behind every GPU number (`oracle_run.py`, `gpu_race.py`, `gpu_heldout.py`) and the SuffixDecoding pass counts (`heldout_suffix.py`, `heldout_suffix2.py`). About 10–12 GPU hours of free Kaggle quota were used in total.

Acknowledgements and AI-assistance statement

The code, experiments and text were produced with Claude (Anthropic), an AI assistant, under the author's direction. The author chose the problem, set the kill rules and approved every result. Every number in this report comes from a logged run in the repository. Thanks to Kortix for releasing the FastApply model and dataset, and to Snowflake for open-sourcing Arctic Inference.

References

1. Yang et al. *Inference with Reference: Lossless Acceleration of Large Language Models* (LLMA). arXiv:2304.04487, 2023.
2. Saxena. *Prompt Lookup Decoding*. GitHub, 2023.
3. Oliaro et al. *SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications*. arXiv:2411.04975; Snowflake Arctic Inference.
4. Wang et al. *EfficientEdit: Accelerating Code Editing via Edit-Oriented Speculative Decoding*. ASE 2025, arXiv:2506.02780.
5. Hu et al. *SAM Decoding: Speculative Decoding via Suffix Automaton*. ACL 2025, arXiv:2411.10666.
6. Shen et al. *Draft Less, Retrieve More: Hybrid Tree Construction for Speculative Decoding* (Graft). arXiv:2605.20104, 2026.
7. He et al. *REST: Retrieval-Based Speculative Decoding*. NAACL 2024.
8. Miao et al. *SpecInfer*. ASPLOS 2024. Cai et al. *Medusa*. ICML 2024.
9. Fu et al. *Break the Sequential Dependency of LLM Inference Using Lookahead Decoding*. ICML 2024.
10. Li, Chen, Hu, Xia. *An Empirical Study of Speculative Decoding on Software Engineering Tasks*. arXiv:2604.26469, 2026.
11. Ke et al. *QiMeng-PRepair: Precise Code Repair via Edit-Aware Reward Optimization*. arXiv:2604.05963, 2026.
12. Cursor. *Editing Files at 1000 Tokens per Second* (blog), 2024.
13. Kortix. *FastApply-1.5B-v1.0* (model) and *FastApply-dataset-v1.0* (dataset), Hugging Face, Apache-2.0.
14. Snowflake. *Arctic Inference* (github.com/snowflakedb/ArcticInference), v0.3.0, Apache-2.0.